



WESTCLIFF
UNIVERSITY

Educate. Inspire. Empower.

DOM & Data Structure Review

Module 5 - Week 4 Day 1



Agenda

1. Sample demo of DOM access
2. Update data in React with Ajax
3. Data Structures: List, Queue, Stack, Tree
4. Sorting data in data structures
5. Asymptotic notation: $O(f(n))$, $o(f(n))$, $\Omega(f(n))$, $\Theta(f(n))$
6. Implement quick sort
7. Lab Assignment
8. Quiz

Sample demo of DOM access



DOM Access

We learned a lot about DOM in previous classes. But today we will learn a bit more about DOM. We will do some exercise and refresh our DOM knowledge.

The **Document Object Model (DOM)** is the data representation of the objects that comprise the structure and content of a document on the web.

DOM represents an HTML or XML document in memory and how you use APIs to create web content and applications. It represents the page so that programs can change the document structure, style, and content.

The DOM represents the document as nodes and objects. The DOM is an object-oriented representation of the web page, which can be modified with a scripting language such as JavaScript.



DOM Access

The modern DOM is built using multiple APIs that work together. The core DOM defines the objects that fundamentally describe a document and the objects within it. This is expanded upon as needed by other APIs that add new features and capabilities to the DOM.

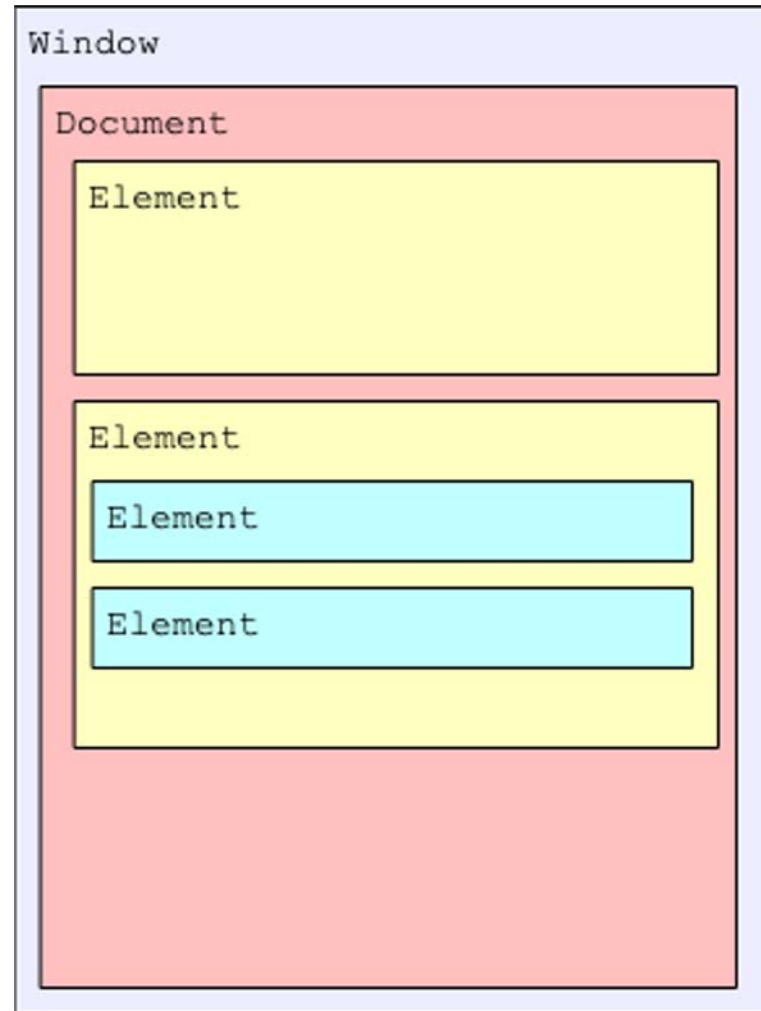
The **HTML DOM API** is an interfaces that define the functionality of each element

- Access to and control of HTML elements via DOM and manipulation of form data.
- Interacting with the contents of 2D images and the context of an HTML <canvas>.
- Management of media connected to HTML elements (<audio> and <video>).
- Dragging and dropping of content on webpages.
- Access to the browser navigation history
- Supporting and connective interfaces for other APIs such as Web Components, Web Storage, Web Workers, WebSocket, and Server-sent events.



DOM Access

Consider a document with two elements, one of which has two more elements nested inside it





DOM Access

The standard DOM specifies that the `querySelectorAll` method in the code below must return a list of all the `<p>` elements in the document:

```
const paragraphs = document.querySelectorAll("p");  
// paragraphs[0] is the first <p> element  
// paragraphs[1] is the second <p> element, etc.  
alert(paragraphs[0].nodeName);
```

All of the properties, methods, and events available for manipulating and creating web pages are organized into objects. The page content is stored in the DOM and may be accessed and manipulated via JavaScript, so that we may write this approximative equation:

API = DOM + JavaScript



DOM Access

When you create a script, whether it's inline in a `<script>` element or included in the web page by means of a script loading instruction. You can immediately begin using the API for the `document` or `window` elements to manipulate the document itself.

Your DOM programming may be something as simple as using the `alert()` function from the window object, or it may use more sophisticated DOM methods to actually create new content, as in the longer example below.

```
<body onload="window.alert('Welcome to my home page!');">
```

On load of the page (since we are using body tag), It will create an `Alert` with a message `Welcome to my home page!`



DOM Access

This function creates a new **H1** element, adds text to that element, and then adds the H1 to the tree for this document **when the page loads**.

```
window.onload = function() {  
    // create a couple of elements in an otherwise empty HTML page  
    const heading = document.createElement("h1");  
    const heading_text = document.createTextNode("Big Head!");  
    heading.appendChild(heading_text);  
    document.body.appendChild(heading);  
}
```



DOM Access - Data Types

Document	Document When a member returns an object of type document. this object is the root document object itself. The DOM document Reference chapter describes the document object.
Node	Every object located within a document is a node of some kind. An object can be an element node but also a text node or attribute node.
Element	The element type is based on node. It refers to an element or a node of type element returned by a member of the DOM API. the <code>document.createElement()</code> method returns an object reference to a node, we just say that this method returns the element that has just been created in the DOM. element objects implement the DOM Element interface and also the more basic Node interface, both of which are included together in this reference.



DOM Access - Data Types

NodeList	A nodeList is an array of elements, like the kind that is returned by the method <code>document.querySelectorAll()</code>. Items in a nodeList are accessed by: • <code>list.item(1)</code> • <code>list[1]</code>
Attribute	When an attribute is returned by a member ex: <code>createAttribute()</code>, it is an object reference that exposes a special interface for attributes. Attributes are nodes in the DOM just like elements are.
NamedNode Map	A <code>namedNodeMap</code> is like an array, but the items are accessed by name or index. A <code>namedNodeMap</code> has an <code>item()</code> method for this purpose, and you can also add and remove items from a <code>namedNodeMap</code>.

A common terminology to keep in mind: It's common to refer to any Attribute node as an attribute.



DOM Interface and Objects

There are many points where understanding how Interface and Objects work can be confusing.

The object representing the HTML `form` element gets its name `property` from the `HTMLFormElement` interface but its `className` property from the `HTMLElement` interface. In both cases, the property you want is in that form object.

Many objects borrow from several different interfaces. The `table` object, for example, implements a specialized `HTMLTableElement` interface, which includes such methods as `createCaption` and `insertRow`. But since it's also an HTML element, `table` implements the `Element` interface.

```
const table = document.getElementById("table");
const tableAttrs = table.attributes;
// Node/Element interface
for (let i = 0; i < tableAttrs.length; i++) {
  // HTMLTableElement interface: border attribute
  if(tableAttrs[i].nodeName.toLowerCase() == "border")
    table.border = "1";
}
// HTMLTableElement interface: summary attribute
table.summary = "note: increased border";
```

Update data in React with Ajax



React and Ajax

React can be used with static data, But a question might be: *How do I populate my React components with data from a server-side API?*

React is a view library and doesn't provide any networking capabilities.

Since React has no networking features, passing server-side data to React is a 2-step process:

1. download API data
2. pass that data to React (as props)

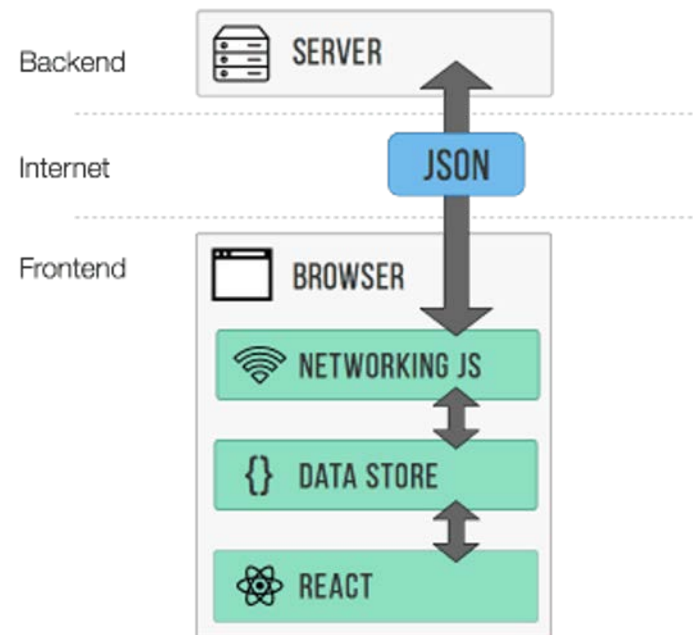


React and Ajax

The flow of data from the server to REACT

You have:

- Server
- JSON
- Browser
 - Networking JS
 - Data Store
 - React



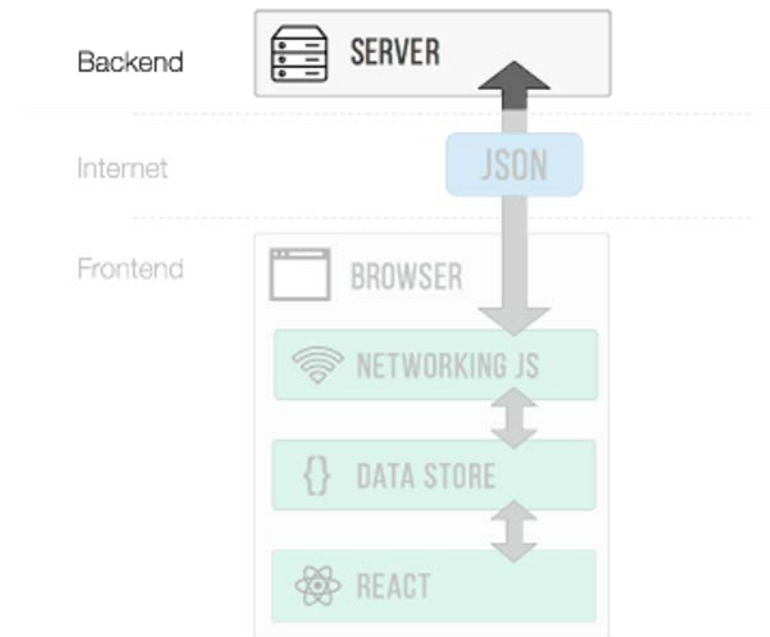


React and Ajax

Server

The server provides data to your application via the internet.

The server could be a 3rd-party API like the GitHub API, or it could be your own server. For your own server you could use Rails, Django, Tomcat, the list goes on. Express is a common choice for React developers because it runs on Node.js.



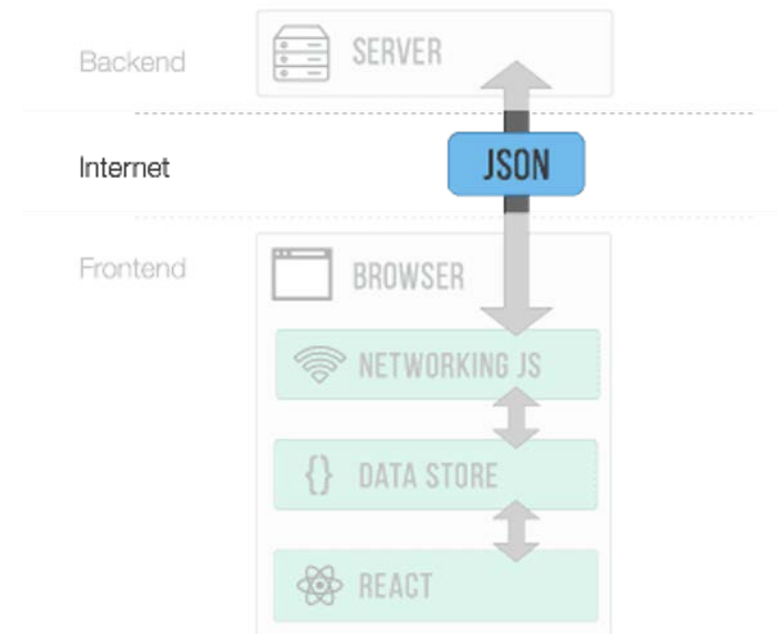


React and Ajax

JSON

The best format for transferring data from the server to the browser is JSON.

JSON is a text-based data format consisting of attribute–value pairs. It looks similar to JavaScript code.





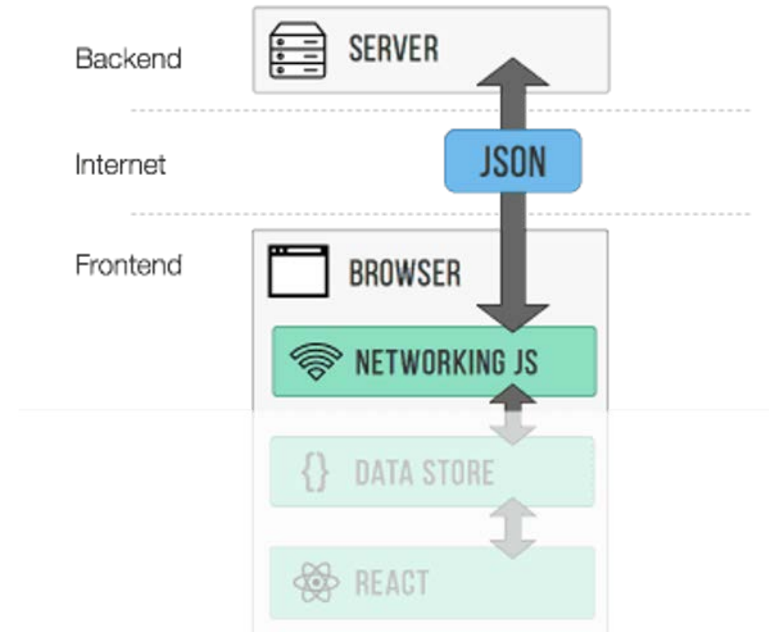
React and Ajax

Networking Javascript

Networking in client-side JavaScript is accomplished with AJAX.

AJAX stands for 'asynchronous javascript and xml'. For React developers this is a bit of a misnomer: JSON will be used much more commonly than XML.

Browsers make AJAX/network requests using the XMLHttpRequest API or the newer Fetch API. There are also many JavaScript libraries that wrap these APIs to provide an easier-to-use interface for you, the developer.





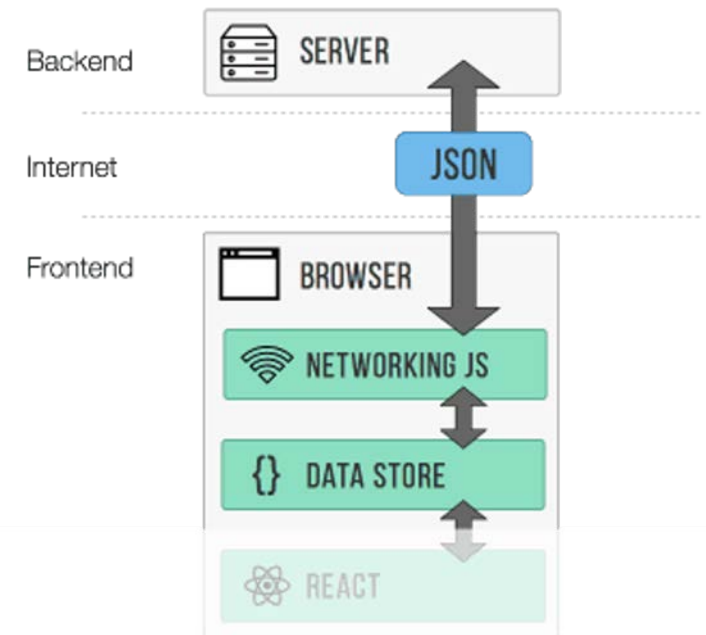
React and Ajax

Data Store

Once you have the data from the server, you'll want to save it somewhere.

The data store could be Redux or another flux.

For simpler application, you could simply store your data in the state of one of your React components.



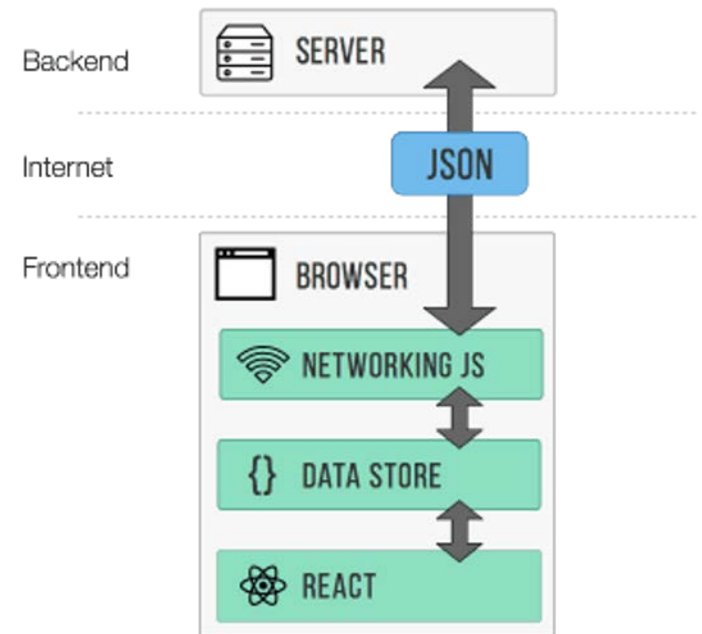


React and Ajax

REACT

Once you have the data, adding it to React completes the picture.

Usually you would populate the data with props. If you use Redux or Relay this will be done for you.





React and Ajax best practices

There are 4 best practices. The approach you use depends on the size and complexity of your app, along with which libraries or technologies you are already using.

These are your options:

1. Root Component
2. Container Components
3. Redux Async Actions
4. Relay

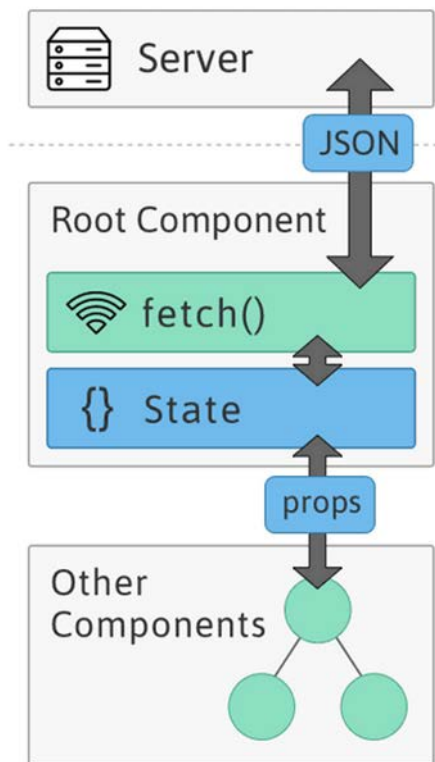


React and Ajax best practices

1. Root Component

This is the simplest approach so it's great for prototypes and small apps.

With this approach, you build a single root/parent component that issues all your AJAX requests. The root component stores the AJAX response data in its state, and passes that state (or a portion of it) down to child components as props.





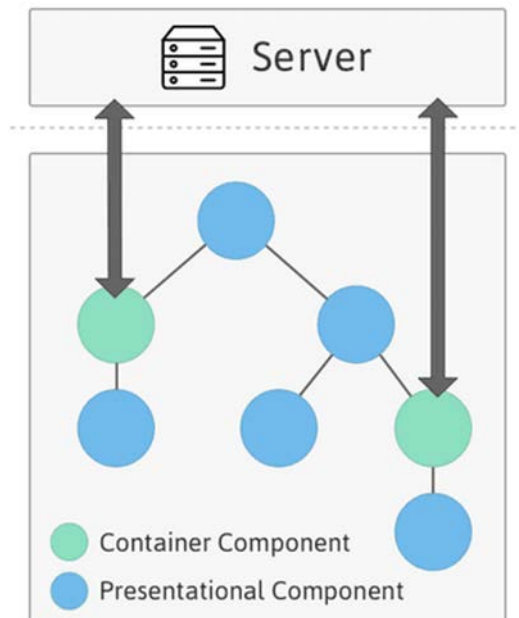
React and Ajax best practices

2. Root Container Components

A container component "provides the data and behavior to presentational or other container components."

The container component approach is just like the root component approach, except with multiple components that can interact with the server.

For every presentational component that needs data from the server, create a container component that sends AJAX requests to get that data, and provides it to the child component via props.





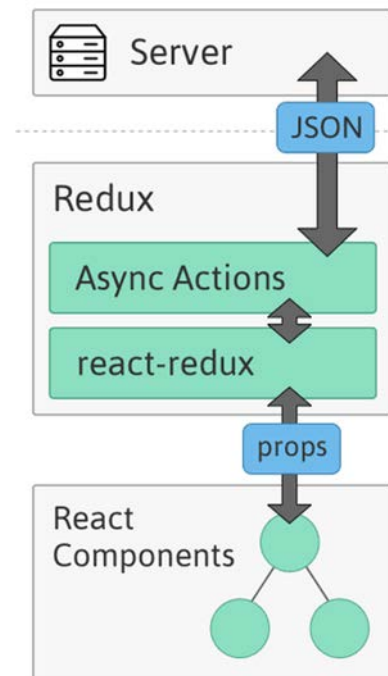
React and Ajax best practices

3. Redux Async Actions

Redux manages data, and AJAX provides data from the server, so it makes sense that your Redux code should be handling your network requests.

If you are using Redux, don't put AJAX in your React components. Instead, put it in your Async Actions.

Using **fetch()** is highly recommended to make the actual network requests.





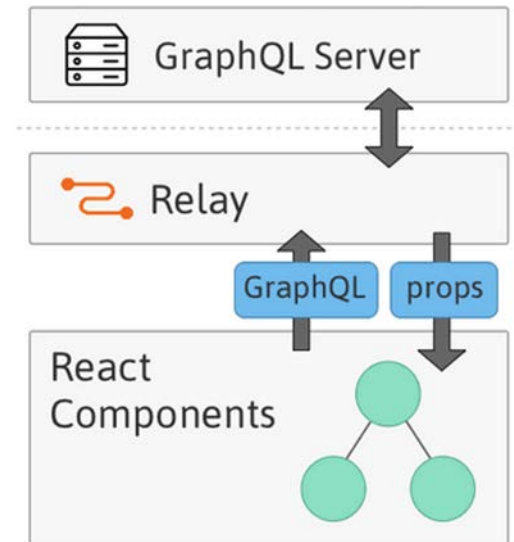
React and Ajax best practices

4. Relay

With Relay, you declare the data needs of your React components with GraphQL, and Relay automatically downloads that data and fills out the component props.

Relay works great for large applications, but requires a large up-front time investment. You need to:

- Learn about Relay and GraphQL.
- Specify the data needs of your React components with GraphQL (instead of propTypes).
- Set up a GraphQL server.



Data Structures: List, Queue, Stack, Tree



Data Structure

Data and its information is an essential part, and various implementations are being made to store in different ways. Data are just a collection of facts and figures, or you can say data are values or a set of values that are in a particular format. A data item refers to a single set of values. Data items are then further categorized into sub-items, which are the group of items that are not being called a plain elementary form of items.

In technical terms, a data structure is a way to store and organize data in a computer's memory so that these data can be used efficiently later. Data may be arranged logical or mathematical model for a particular organization known as a data structure. The data model depends on the two factors:

- The structure should reflect the relationships of the data with the real-world object.
- The structure should be simple enough so that anyone can efficiently process the data each time it is necessary.



Data Structure

The data structure can be subdivided into 2 major types:

- Linear Data Structure
- Non-linear Data Structure



Data Structure

Linear Data Structure

A data structure is a linear if its elements combine to form any specific order. There are two techniques of representing such linear structure within memory.

- The first way is to provide the linear relationships among all the elements represented using linear memory location. These linear structures are termed as arrays.
- The second technique is to provide a linear relationship among all the elements represented by using the concept of pointers or links. These linear structures are termed as linked lists.



Data Structure

Linear Data Structure (continued)

Examples of the linear data structure are:

- Arrays
- Queues
- Stacks
- Linked lists



Data Structure - Linear Structure

Arrays

The array is a fixed-size sequenced collection of variables belonging to the same data types. The array has adjacent memory locations to store values. Arrays are particularly helpful for making a collection of input data which arrive in random order.

Why Arrays? When a program works with many variables which hold comparable forms of data, then organizational and managerial difficulty quickly arise. By not using arrays, the number of variables used will increase.

Element: Every item stored in an array is termed as an element

Index: each memory location of an element in an array is denoted by a numerical index which is used for identifying the element



Data Structure - Linear Structure

Arrays (Continued)

There is some specific operation that can be performed or those that are supported by the array. These are:

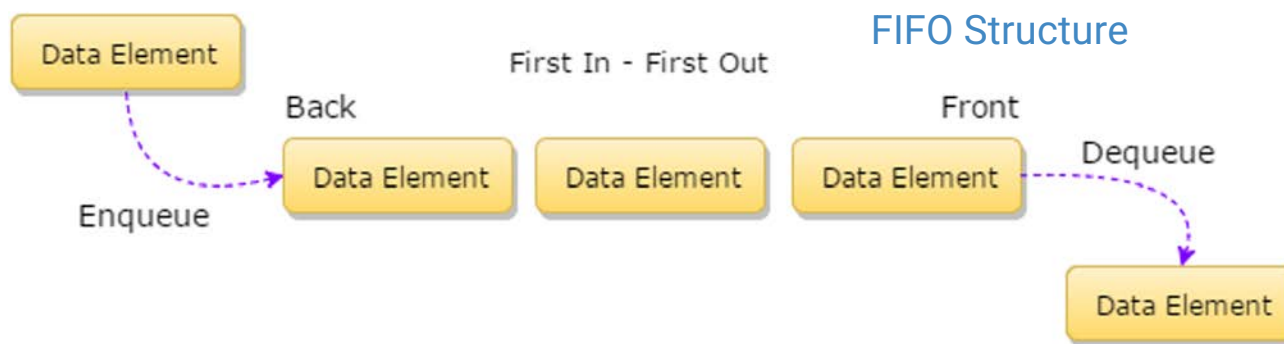
- Traversing: It prints all the array elements one after another.
- Inserting: It adds an element at given index.
- Deleting: It is used to delete an element at given index.
- Searching: It searches for an element(s) using given index or by value.
- Updating: It is used to update an element at given index.



Data Structure - Linear Structure

Queues

The queue is a linear data structure used to represent a linear list. It allows insertion of an element to be done at one end and deletion of an element to be performed at the other end. The deletion of an element can take place only at one end called the front and insertion can take place on the other end which is termed as the rear. The term front and rear are frequently used while describing queues in a linked list.





Data Structure - Linear Structure

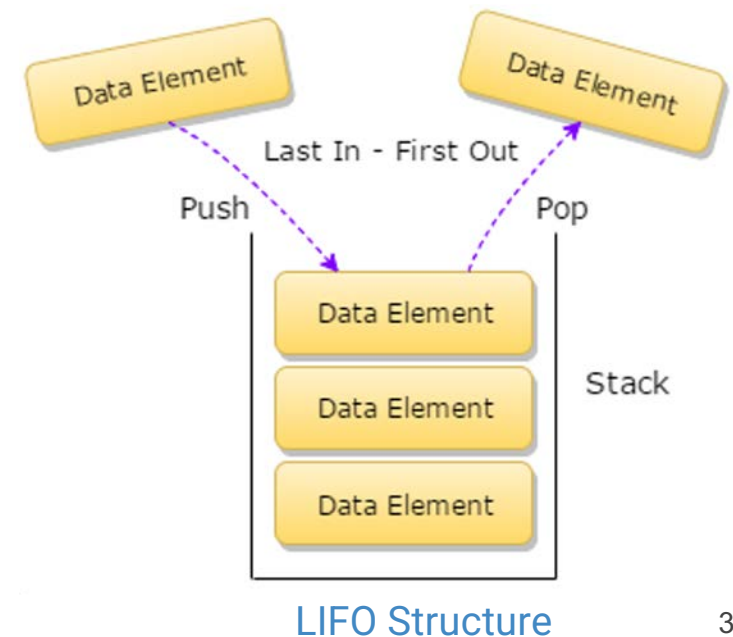
Stacks

The Stacks usually falls under the category of an abstract data type which serves as a concrete and valuable tool for problem-solving.

A stack is a linear data structure in which all the insertion and deletion of data or you can say its values are done at one end only, rather than in the middle. Stacks can be implemented by using arrays of type linear.

The stack is mostly used in converting and evaluating expressions in Polish notations:

- Infix
- Prefix
- Postfix





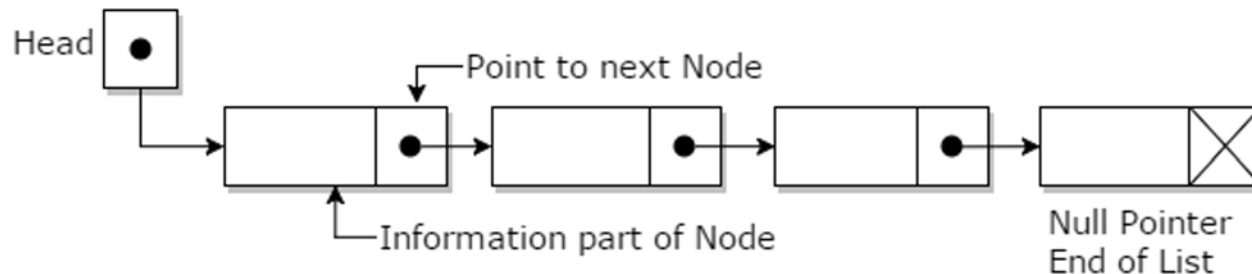
Data Structure - Linear Structure

Linked Lists

The linked list or one way list is a linear set of data elements which is also termed as nodes. Here, the linear order is specified using pointers.

Each node is separated into two different parts:

- The first part holds the information of the element or node
- The second piece contains the address of the next node (link / next-pointer field) in this structure list.





Data Structure - Linear Structure

Linked Lists (Continued)

Linked lists can be measured as a form of high-level standpoint as being a series of nodes where each node has at least one single pointer to the next connected node, and in the case of the last node, a null pointer is used for representing that there will be no further nodes in the linked list.

In the data structure, you will be implementing the linked lists which always maintain head and tail pointers for inserting values at either the head or tail of the list is a constant time operation. Linked lists in data structure have some characteristics:

- Insertion is $O(1)$
- Deletion is $O(n)$
- Searching is $O(n)$



Data Structure - Non-Linear Structure

Non-Linear Data Structure

This structure is mostly used for representing data that contains a hierarchical relationship among various elements.

Examples of Non-Linear Data Structures:

- Graphs
- The family of trees and
- Table of contents



Data Structure - Non-Linear Structure

Non-Linear Data Structure (Continued)

Tree: In this case, data often contain a hierarchical relationship among various elements. The data structure that reflects this relationship is termed as a rooted tree graph or a tree.

Graph: In this case, data sometimes hold a relationship between the pairs of elements, which is not necessarily following the hierarchical structure. Such a data structure is termed as a Graph.

Sorting data in data structures



Sorting Data

Sorting refers to the operation or technique of arranging and rearranging sets of data in some specific order. A collection of records called a list where every record has one or more fields. The fields which contain a unique value for each record is termed as the **key** field. Sorting method can be implemented in different ways - by selection, insertion method, or by merging.

Example: Phone Directory, it is sorted by name, address

Sorting is the operation performed to arrange the records of a table or list in some order according to some specific ordering criterion. Sorting is performed according to some key value of each record. The records are either sorted either numerically or alphanumerically. The records are then arranged in ascending or descending order depending on the numerical value of the key.



Sorting Data

The techniques of sorting can be divided into two categories.

- **Internal Sorting:** If all the data that is to be sorted can be adjusted at a time in the main memory, the internal sorting method is being performed.
- **External Sorting:** When the data that is to be sorted cannot be accommodated in the memory at the same time and some has to be kept in auxiliary memory such as hard disk, floppy disk, magnetic tapes etc, then external sorting methods are performed.



Sorting Data

Complexity of Sorting Algorithms

The complexity of sorting algorithm calculates the running time of a function in which ' n ' **number of items** are to be sorted. The choice for which sorting method is suitable for a problem depends on several dependency configurations for different problems. The most noteworthy of these considerations are:

- The length of time spent by the programmer in programming a specific sorting program
- Amount of machine time necessary for running the program
- The amount of memory necessary for running the program



Sorting Data

Efficiency of Sorting

The amount of time required to sort an array of ' n ' elements by a particular method, the normal approach is to analyze the method to find the number of comparisons required by it. Most of the sorting techniques are data sensitive, and so the metrics for them depends on the order in which they appear in an input array.

Various sorting techniques are analyzed in various cases:

- Best case
- Worst case
- Average case



Sorting Data

Types of Sorting Techniques

- Bubble Sort
- Selection Sort
- Merge Sort
- Insertion Sort
- Quick Sort
- Heap Sort



Sorting Data

Types of Sorting Techniques

- **Bubble Sort:** Bubble Sort Algorithm is used to arrange **N elements** in ascending order, and for that, you have to begin with 0th element and compare it with the first element. If the 0th element is found greater than the 1st element, then the swapping operation will be performed, i.e., the two values will get interchanged. In this way, all the elements of the array get compared.
- **Selection Sort:** The selection is a straightforward process of sorting values. In this method, to sort the data in ascending order, the 0th element is compared with all other elements. If the 0th element is found to be greater than the compared element, the two values get interchanged. In this way after the first iteration, the smallest element is placed at 0th position. The technique is repeated until the full array gets sorted.



Sorting Data

Types of Sorting Techniques

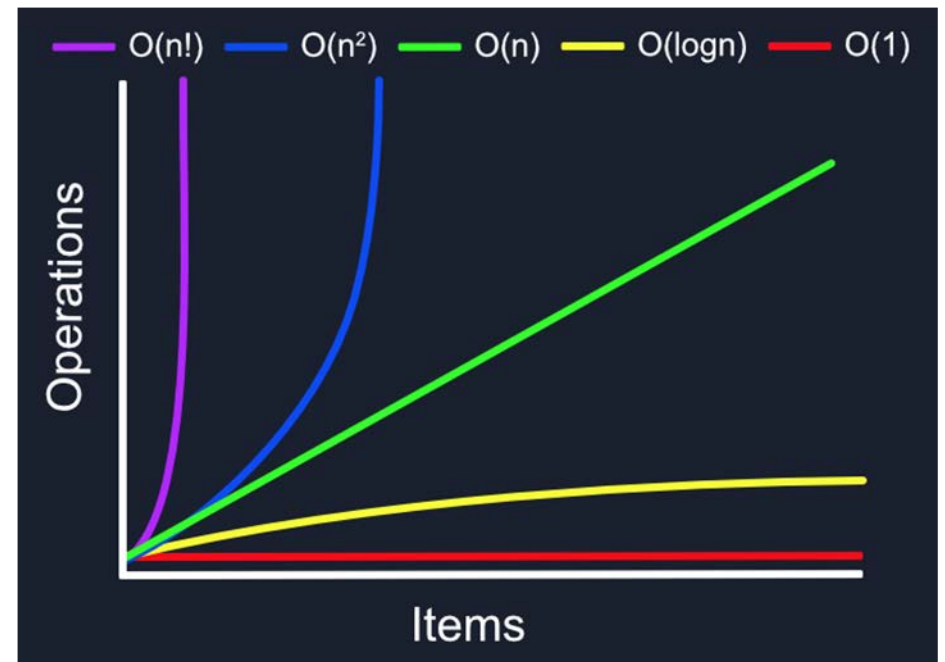
- **Merge Sort:** Merge sort is another sorting technique and has an algorithm that has a reasonably proficient space-time complexity - $O(n \log n)$ and is quite trivial to apply. This algorithm is based on splitting a list, into two comparable sized lists, i.e., left and right and then sorting each list and then merging the two sorted lists back together as one.

Asymptotic notation:
 $O(f(n))$, $o(f(n))$, $\Omega(f(n))$, $\Theta(f(n))$



Asymptotic Notation

Also known as **Big O Notation**. Big O notation is just a way of representing the general growth in the computational difficulty of a task as you increase the data set. While there are other notations, O notation is generally the most used because it focuses on the worst-case scenario, which is easier to quantify and think about. Worst-case meaning where the most operations are needed to complete the task.





Asymptotic Notation

$O(1)$

This is the ideal, no matter how many items there are, whether one or one million, the amount of time to complete will remain the same. Most operations that perform a single operation are $O(1)$. Pushing to an array, getting an item at a particular index, adding a child element, etc, will all take the same amount of time regardless of the array length.

$O(1)$ does not change with respect to input space. Hence, $O(1)$ is referred to as being constant time.

```
const a1 = performance.now();  
smArr.push(27);  
  
const a2 = performance.now();  
console.log(`Time: ${a2 - a1}`); // Less than 1 Millisecond
```



Asymptotic Notation

$O(n)$

By default, all loops are an example of linear growth because there is a one-to-one relationship between the data size and time to completion. So an array with 1,000 times more items will take exactly 1,000 times longer.

```
const a1 = performance.now();  
smArr.forEach(item => console.log(item));
```

```
const a2 = performance.now();  
console.log(`Time: ${a2 - a1}`); // 3 Milliseconds
```



Asymptotic Notation

$O(n^2)$

Exponential growth is a trap we've all fall into at least once. Need to find a matching pair for each item in an array? Putting a loop inside a loop is great way of turning an array of 1,000 items into a million operation search that'll freeze your browser. It's always better to have to do 2,000 operations over two separate loops than a million with two nested loops.

```
const a1 = performance.now();
smArr.forEach(() => {
  arr2.forEach(item => console.log(item));
});
```

```
const a2 = performance.now();
console.log(`Time: ${a2 - a1}`); // 8 Milliseconds
```



Asymptotic Notation

$O(\log n)$

To understand the $O(\log n)$ we will use the example of a dictionary. The amount of time to find something will still change depending on the size of the dictionary but at nowhere near the rate of $O(n)$. Because it searches in progressively more specific sections without looking at most of the data, a search through a thousand items may take less than 10 operations while a million may take less than 20.

```
const a1 = performance.now();
smArr.forEach(() => {
  arr2.forEach(item => console.log(item));
});
```

```
const a2 = performance.now();
console.log(`Time: ${a2 - a1}`); // 8 Milliseconds
```



Asymptotic Notation

$O(n!)$

To understand the $O(n!)$ we will use the travelling salesman problem. If you have a bunch of cities of varying distance, how do you find the shortest possible route that goes between all of them and returns to the starting point? The brute force method would be to check the distance between every possible configuration between each city, which would be a factorial and quickly get out of hand.

We'll demonstrate this complexity with a short recursive function. This function will multiply a number by its own function taking in itself minus one. Every digit in our factorial will run its own function until it reaches 0, with each recursive layer adding its product to our original number. So 3 is multiplied by 2 that runs the function to be multiplied by 1 that runs it again to be stopped at 0, returning 6. Recursion gets confusing like this pretty easily.



Asymptotic Notation

$O(n!)$

Example:

```
const factorial = n => {  
  let num = n;  
  
  if (n === 0) return 1  
  for (let i = 0; i < n; i++) {  
    num = n * factorial(n -  
1);  
  };  
  
  return num;  
};
```



Asymptotic Notation - Rules

Let's represent an algorithm's complexity as $f(n)$. n represents the number of inputs, $f(n)_{\text{time}}$ represents the time needed, and $f(n)_{\text{space}}$ represents the space (additional memory) needed for the algorithm. The goal of algorithm analysis is to understand the algorithm's efficiency by calculating $f(n)$.

However, it can be challenging to calculate $f(n)$. Big-O notation provides some fundamental rules that help developers compute for $f(n)$.

Implement quick sort



Quick Sort

Quick sort is one of the most famous sorting algorithms based on divide and conquers strategy which results in an $O(n \log n)$ complexity. So, the algorithm starts by picking a single item which is called pivot and moving all smaller items before it, while all greater elements in the later portion of the list. This is the main quick sort operation named as a partition, recursively repeated on lesser and greater sublists until their size is one or zero - in which case the list is wholly sorted.

There are many different versions of quickSort that pick pivot in different ways.

1. Always pick first element as pivot.
2. Always pick last element as pivot
3. Pick a random element as pivot.
4. Pick median as pivot.



Quick Sort

The key process in quickSort is `partition()`. Target of partitions is, given an array and an element x of array as pivot, put x at its correct position in sorted array and put all smaller elements (smaller than x) before x , and put all greater elements (greater than x) after x . All this should be done in linear time.

Partition Algorithm

There can be many ways to do partition, following pseudo code adopts the method given in CLRS book. The logic is simple, we start from the leftmost element and keep track of index of smaller (or equal to) elements as i . While traversing, if we find a smaller element, we swap current element with `arr[i]`. Otherwise we ignore current element.

Resources

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction

<https://reactjs.org/docs/refs-and-the-dom.htm>

<https://www.javascriptstuff.com/how-ajax-works-react/>

<https://www.w3schools.in/data-structures-tutorial/sorting-techniques/>

<https://www.cs.yale.edu/homes/aspnes/pinewiki/AsymptoticNotation.html>

<https://www.digitalocean.com/community/tutorials/js-big-o-notation>

The background is a solid light blue color. In the bottom-left corner, there is a decorative geometric pattern consisting of several overlapping triangles and squares in different shades of blue, ranging from a very dark navy blue to a medium blue.

Questions?